One-sided transport for concise iPRA

A seven-rule presentation with one numerical equality template

Research note prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

The current ConciseIPRA development already has a seven-rule presentation of full first-order intuitionistic primitive recursive arithmetic. This note simplifies its equality inference: replace two varying numerical templates by one, using

$$\frac{s = t \qquad f(s) = r}{r = f(t)} \qquad (s \not\equiv_{\mathrm{syn}} 0).$$

Two applications recover the old equational transport rule. Conversely, ordinary equational transport followed by symmetry derives this rule. All numerical syntax, both formula binders, ordinary closure elimination, and the single introduction rule remain as in the current seven-rule calculus. The new presentation derives exactly the same sequents, including open formulas under arbitrary assumptions. Its interpretations in iPRA therefore retain the whole numerical domain, actual equality, and both identity round trips.

There is also a useful strengthening: the zero computation equation and the new transport rule alone recover the entire equality calculus, including transport with a literal-zero source. This avoids arithmetic induction and any operation reversing compound formulas.

This is a proved simplification of the mathematical inference interface. It is not a proof of absolute minimality or a new record for complete Python checker size. In matched standalone implementations with full syntax and scope validation, it saves 25 significant tokens, 25 AST nodes, and 56 normalized bytes (about 1.7 percent). Its measurements and limits are stated separately. A proposed Lean extension is included but has not been compiled.

Contents

1 The result and the comparison target 3

2 Complete primitive presentation 4
2.1 Syntax, binding, and interpretations . . . 4
2.2 The seven rules . . . 5

3 The equality calculus from I_0 and X alone 5

4 Equivalence of the complete calculi 7
4.1 A short proof through the existing seven-rule calculus . . . 7
4.2 Direct recovery of the native rules . . . 7
4.3 What this equivalence preserves . . . 8

5 The full intuitionistic logical interface 8
5.1 Finite formula selectors . . . 8
5.2 Connective and quantifier macros . . . 9

6 Bi-interpretation with full first-order iPRA 9
6.1 The standard target theory . . . 9
6.2 The inherited scalar bridge . . . 10
6.3 The two translations explicitly . . . 10

7 Implementation, measurements, and validation 11
7.1 Scope of the executable evidence . . . 11
7.2 Measured compact baselines . . . 12
7.3 Lean status . . . 12

8 Alternatives screened out and further research questions 13

A Reproduction and archive inventory 13

1 The result and the comparison target

The comparison is pinned to ProveIt commit `8eb71c1e570e07ef45ead686628e77f18ef34798`, inspected on November 11, 2026 in the user's time zone. The relevant directory is `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The root README, native rules, seven-rule calculus, and bi-interpretation theorems were read directly from this snapshot[1, 2, 3, 4].

Write $\mathcal{M}$ for the existing seven-rule calculus `MPrf`. Its transport rule is

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash a(s) = b(s)}{\Gamma \vdash a(t) = b(t)} \quad (s \not\equiv_{\text{syn}} 0). \tag{Old Tr}$$

Here $a(v)$ and $b(v)$ are independently chosen numerical templates. The proposed calculus $\mathcal{C}$, represented by `CPrf` in the accompanying Lean source, replaces this rule by

$$\boxed{\frac{\Gamma \vdash s = t \qquad \Gamma \vdash f(s) = r}{\Gamma \vdash r = f(t)} \quad (s \not\equiv_{\text{syn}} 0).} \tag{X}$$

Only $f(v)$ is a numerical template. The endpoint r is an ordinary term in the ambient context; the template variable is not bound in it. Constant templates are allowed and are essential.

The central calculation is short:

$$a(s) = b(s) \xrightarrow[s=t]{X,\ f=a} b(s) = a(t) \xrightarrow[s=t]{X,\ f=b} a(t) = b(t). \tag{Two steps}$$

The source restriction remains $s \not\equiv_{\text{syn}} 0$ at both steps. No arithmetic or logical theorem is used in this simulation.

Theorem 1.1 (Main comparison)**.** *For every finite context Γ and native formula A,*

$$\Gamma \vdash_{\mathcal{C}} A \iff \Gamma \vdash_{\mathcal{M}} A \iff \Gamma \vdash_{\mathcal{N}} A,$$

where $\mathcal{N}$ is the current eight-rule native scalar core `NPrf`. Consequently $\mathcal{C}$ is bi-interpretable and equiconsistent with full first-order iPRA, with the inherited whole-domain interpretations and identity graphs $x = y$ for the numerical composites.

The precise simplification is a decrease from two varying numerical endpoints to one in the primitive equality rule. It is a restriction of the previously studied crossed transport when that rule is applied to an equation with one constant side[6]. It is not literally a subset of the old *uncrossed* rule: the conclusion has been reversed. Unlike the earlier general reversal presentation, $\mathcal{C}$ needs no reversal operation on arbitrary native formulas and does not modify QE.

Feature	Existing $\mathcal{M}$	Proposed $\mathcal{C}$
Primitive inference schemas	7	7
Varying numerical templates in transport	2	1
Template instances in the displayed transport	4	2
Transport source may be the literal 0	No	No
Ordinary QE	Yes	Yes
Whole native language preserved	Yes	Yes

The ordinary term r remains part of the conclusion and must be validated. Thus the table does not by itself assert a proportional reduction in program size, certificate size, or runtime. The existing small implementations share machinery between QE and two-sided transport; this sharing can outweigh the simpler standalone equality schema.

2 Complete primitive presentation

2.1 Syntax, binding, and interpretations

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the update body h binds u; the counter n and initializer a are outside that binding. All terms and formulas are finite syntax. Variables never range over program descriptions, and there is no uniform primitive-recursive evaluator.

The formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

In $Q_y B$ the variable y is bound. In $D_{x,y} B$ both x and y are bound and distinct. Bound variables are renamed to avoid capture. Their ordinary first-order readings are

$$\begin{aligned} (s = t)^\circ &= (s = t), \\ (Q_y B)^\circ &= B^\circ(0) \to \forall y\, B^\circ(y), \\ (D_{x,y} B)^\circ &= \exists x \,\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big). \end{aligned}$$

These are fixed readings at every matrix, including equations. An atomic D does not receive a different, progressive semantics.

A context Γ is a finite reusable list of native formulas. The designated contradiction is $0 = S0$, also written $0 = 1$. The sign $s \not\equiv_{\text{syn}} 0$ means that s is not the *literal zero term*; it does not mean that its numerical value is nonzero. In particular, $I(0, 0; u.0) \not\equiv_{\text{syn}} 0$.

For a fixed matrix $B(x, y)$ and witness term t, set

$$\eta_{B,t}(z) = \begin{cases} z, & B \text{ is a literal equation and } t \equiv_{\text{syn}} 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Start}$$

"Literal equation" tests the outermost formula constructor. An equation inside Q or D is not a literal equation at that outer level.

2.2 The seven rules

The following is the entire primitive inventory. Every rule retains the displayed context Γ; local assumptions are discharged.

$$\frac{}{\Gamma \vdash A} \quad (A \in \Gamma), \qquad \text{Hyp},$$

$$\frac{}{\Gamma \vdash I(0, a; u.h) = a}, \qquad I_0,$$

$$\frac{}{\Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]}, \qquad I_S,$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash f(s) = r}{\Gamma \vdash r = f(t)} \quad (s \not\equiv_{\mathrm{syn}} 0), \qquad X,$$

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}, \qquad \text{QE},$$

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B(b, y) \vdash A}{\Gamma \vdash A}, \qquad \text{DE},$$

$$\frac{\Gamma, B(t, \eta_{B,t}(z)) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B(x, y)}, \qquad \text{DI}.$$

In DE, b is fresh for Γ, A, and the passive parameters of B. In DI, z is fresh for Γ, t, and the passive parameters of B. In X, the template $f(v)$ is a numerical term with one designated substitution variable v, chosen fresh for Γ, s, t, r; it may use that variable zero, one, or several times. All substitutions avoid capture.

There are no additional primitive equality rules, logical connectives, logical axioms, successor axioms, coverage axioms, or induction schemas. Weakening, exchange, contraction, fresh renaming, and ordinary-assumption cut are admissible by induction on derivations. Fresh variables are renamed before a cut is inserted. These operations do not change whether a source or a witness is the literal zero term.

Arbitrary substitution into a derivation is not used in the following bootstrap argument: it can change a syntactically nonzero source into 0, or change the DI start choice. After Theorem 1.1, substitution is admissible through the equivalent native calculus. What fails directly is a constructor-preserving transformation of every restricted rule instance, not extensional admissibility of substitution.

3 The equality calculus from I_0 and X alone

All derivations in this section are over an arbitrary fixed Γ. Put

$$J(a) := I(0, a; u.0), \qquad c := J(0).$$

The equation $J(a) = a$ is an instance of I_0. The terms $J(a)$ and c are syntactically nonzero regardless of a.

Lemma 3.1 (Reflexivity and symmetry)**.** *Every equation $a = a$ is derivable. From $a = b$ one can derive $b = a$, including when either endpoint is the literal zero term.*

Proof. For reflexivity use $J(a) = a$ twice in X, with template $f(v) = v$. The conclusion is $a = a$.

For symmetry use the major premise $c = 0$, supplied by I_0, the constant template $f(v) = a$, and the given minor premise $a = b$. The conclusion of X is $b = a$. The source condition holds because $c \not\equiv_{\text{syn}} 0$. □

This proof explains both the utility of crossing the conclusion and the need to permit a constant template. Forbidding unused template variables would remove the displayed symmetry construction.

Lemma 3.2 (Restricted two-sided transport)**.** *If $s \not\equiv_{\text{syn}} 0$, then $s = t$ and $a(s) = b(s)$ imply $a(t) = b(t)$.*

Proof. Apply X first with template a and passive endpoint $b(s)$, then with template b and passive endpoint $a(t)$, as in (Two steps). Both majors are the same equation $s = t$. □

Lemma 3.3 (Joining through zero)**.** *If $p, q \not\equiv_{\text{syn}} 0$, then $p = 0$ and $q = 0$ imply $p = q$.*

Proof. Apply X to $p = 0$ and $J(p) = p$, using $f = J$. This gives $p = c$. The same construction gives $q = c$. Symmetry gives $c = p$ and $c = q$. Finally use X with major $c = q$, minor $c = p$, and $f(v) = v$. Its conclusion is $p = q$. Every source is one of p, q, c and hence is syntactically nonzero. □

Lemma 3.4 (Unrestricted transitivity)**.** *From $r = s$ and $s = t$ one can derive $r = t$.*

Proof. If $s \not\equiv_{\text{syn}} 0$, first reverse $r = s$ and then apply X to major $s = t$ and minor $s = r$, with the identity template.

Suppose $s \equiv_{\text{syn}} 0$. If $r \equiv_{\text{syn}} 0$, the second premise is the result. If $t \equiv_{\text{syn}} 0$, the first premise is the result. Otherwise $r, t \not\equiv_{\text{syn}} 0$, and the premises, after reversing the second, are $r = 0$ and $t = 0$. Lemma 3.3 applies. □

The case distinctions here concern finite syntax in the metaproof. They do not invoke excluded middle or an induction axiom in the object theory.

Lemma 3.5 (Unrestricted numerical congruence)**.** *For every numerical template $f(v)$, $s = t$ implies $f(s) = f(t)$.*

Proof. If $s \not\equiv_{\text{syn}} 0$, apply X with the minor premise $f(s) = f(s)$ from reflexivity. If $s \equiv_{\text{syn}} 0$ and $t \not\equiv_{\text{syn}} 0$, reverse the major, apply the preceding construction, and reverse the result. If both endpoints are literally 0, use reflexivity. □

Theorem 3.6 (Full equational transport)**.** *Without any source restriction, $s = t$ and $a(s) = b(s)$ imply $a(t) = b(t)$.*

Proof. Congruence, symmetry, and two transitivity steps give the chain

$$a(t) = a(s) = b(s) = b(t).$$

All ingredients were obtained using I_0 and X alone. □

This also covers substitution into iterator bodies: f is an arbitrary finite numerical template, and capture-free substitution renames an iterator's state binder when needed. No extensionality or evaluation of program descriptions is being assumed.

4 Equivalence of the complete calculi

4.1 A short proof through the existing seven-rule calculus

Every rule of $\mathcal{M}$ other than transport is literally a rule of $\mathcal{C}$. Lemma 3.2 simulates its restricted transport. Induction on finite derivations therefore gives

$$\Gamma \vdash_{\mathcal{M}} A \implies \Gamma \vdash_{\mathcal{C}} A.$$

Conversely, the native core $\mathcal{N}$ derives X: use its equational transport with templates $f(v)$ and the constant r to obtain $f(t) = r$, and then use native symmetry. Its single-introduction theorem admits DI exactly as stated above. Thus

$$\Gamma \vdash_{\mathcal{C}} A \implies \Gamma \vdash_{\mathcal{N}} A.$$

The existing theorem `mprf_iff_nprf` gives $\mathcal{N} \Rightarrow \mathcal{M}$[3]. These three implications prove the comparison in Theorem 1.1.

This shortest proof needs neither a new arithmetic bridge nor a new proof of the inherited single-introduction theorem. The following direct argument makes the latter dependency explicit and independently explains why the start convention does not lose any native rule.

4.2 Direct recovery of the native rules

The current eight-rule core $\mathcal{N}$ has Hyp, I_0, I_S, unrestricted two-sided equational transport, QE, DE, and these two introductions:

$$\frac{\Gamma, B(t,0) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}\ \text{DI-case}, \qquad \frac{\Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash Q_y E(y)}\ \text{QI-progress}, \qquad \text{(Native introductions)}$$

where E must be a literal equation and the usual fresh-variable conditions hold[2].

Theorem 3.6 already supplies unrestricted transport in $\mathcal{C}$. To recover QI-progress, choose a fresh, unused witness variable x. DI with matrix $E(y)$ and literal witness 0 takes exactly the premise $E(z) \vdash E(Sz)$ and concludes $D_{x,y}E(y)$. DE opens this at a fresh witness, whose absence from E leaves precisely $Q_y E(y)$. The minor premise of DE is Hyp. Thus QI-progress is admissible without equality reasoning.

DI-case agrees directly with DI unless its matrix is an equation and its witness is literally 0. In that exceptional case write the matrix as $E(x,y)$ and suppose

$$\Gamma, E(0,0) \vdash E(0,Sz).$$

Use instead the witness $c = I(0,0;u.0) \not\equiv_{\text{syn}} 0$. Under the assumption $E(c,0)$, equational transport along $c = 0$ gives $E(0,0)$. Insert the displayed premise proof by ordinary-assumption cut, obtaining $E(0,Sz)$. Reverse $c = 0$ and use equational transport again to obtain $E(c,Sz)$. At the witness c, the start is 0, so DI proves the original formula $D_{x,y}E(x,y)$. The witness changes only inside the derivation; the conclusion is identical.

This proves $\mathcal{N} \Rightarrow \mathcal{C}$ directly. For the converse, only DI needs explanation beyond the old transport-plus-symmetry derivation of X. Its case instances are already native DI-case. In a progress instance, set $E(y) = B(0,y)$. Native QI-progress gives $Q_y E(y)$. Under the base assumption $E(0)$, QE supplies $E(Sz)$, and native DI-case with witness 0 concludes $D_{x,y}B$. Thus every DI instance is native-admissible.

4.3 What this equivalence preserves

The translations between $\mathcal{C}$, $\mathcal{M}$, and $\mathcal{N}$ are identity maps on terms, formulas, contexts, and conclusions. They transform derivations. They preserve arbitrary passive numerical parameters and arbitrary retained assumptions, including compound ones. Hence the result is stronger than agreement on closed equations or arithmetic sentences.

Full formula transport, contextual equation induction, and every derived logical rule transfer from the native core. They are not extra primitive rules of $\mathcal{C}$. In particular, no induction at an arbitrary compound formula is inserted.

Two uses of X need two uses of the major proof of $s = t$. A proof DAG may share that proof, but a tree certificate may duplicate it. The two-step calculation does not establish a linear bound on serialized tree certificates, and no such bound is claimed here.

5 The full intuitionistic logical interface

The logical interface is inherited, not a second proposed change. It is recorded explicitly to identify which full first-order theory is obtained. Let

$$C(v, a, b) := I(v, a; u.b).$$

Computation and equality give $C(0, a, b) = a$ and $C(Sz, a, b) = b$. The bound state u is unused in this abbreviation.

5.1 Finite formula selectors

Here are explicit recursive definitions matching the native development [7]. In the following displays every newly bound variable is fresh for the control and passive terms, and those terms are implicitly weakened under the displayed binders. First define an equation-to-formula selector $\mathrm{L}(v; s, t; B)$:

$$\begin{aligned}
\mathrm{L}(v; s, t; u = w) &:= \big(C(v, s, u) = C(v, t, w)\big),\\
\mathrm{L}(v; s, t; Q_y B) &:= Q_y\, \mathrm{L}(v; C(y, 0, s), C(y, 0, t); B),\\
\mathrm{L}(v; s, t; D_{x,y} B) &:= D_{x,y}\, \mathrm{L}(v; C(y, 0, s), C(y, 0, t); B).
\end{aligned}$$

Put $\mathrm{G}_v\, A := \mathrm{L}(v; 0, 0; A)$. Define a selector between arbitrary native formulas by

$$\begin{aligned}
\mathrm{Sel}(v, s = t, B) &:= \mathrm{L}(v; s, t; B),\\
\mathrm{Sel}(v, Q_y A, B) &:= Q_y\, \mathrm{Sel}(v, A, \mathrm{G}_y\, B),\\
\mathrm{Sel}(v, D_{x,y} A, B) &:= D_{x,y}\, \mathrm{Sel}(v, A, \mathrm{G}_y\, B).
\end{aligned}$$

These are operations on finite formula syntax, not new function symbols or quantification over formulas. The first recursion decreases the depth of B; the second decreases the depth of A.

The native endpoint theorems, transferred by exact sequent equivalence, give

$$\vdash_{\mathcal{C}} \mathrm{G}_0\, A, \qquad \mathrm{G}_{Sz}\, A \dashv\vdash_{\mathcal{C}} A, \qquad \mathrm{Sel}(0, A, B) \dashv\vdash_{\mathcal{C}} A, \qquad \mathrm{Sel}(Sz, A, B) \dashv\vdash_{\mathcal{C}} B.$$

Their proofs use binder congruence and padding a formula with a closure whose zero instance is true. They do not assume a Boolean value for an arbitrary quantified formula.

5.2 Connective and quantifier macros

Let $p(n) = I(n, 0; u.C(u, 1, 0))$, so computation gives $p(1) = 1$ and $p(2) = 0$. Define the ordinary logical notation as the following macros:

$$\begin{aligned} \bot_* &:= (0 = 1), \\ A \to_* B &:= Q_y \operatorname{Sel}(y, A, B), \\ A \wedge_* B &:= Q_y \operatorname{G}_y \operatorname{Sel}(p(y), A, B), \\ \exists_* x\, A(x) &:= D_{x,y} \operatorname{G}_y A(x), \\ A \vee_* B &:= \exists_* v\ \operatorname{Sel}(v, A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* Q_x A(x). \end{aligned}$$

All auxiliary variables are fresh. Negation is $A \to_* \bot_*$. These formulas have the full ordinary intuitionistic readings, up to provable equivalence, under the realization in Section 2.

For clarity, the derived ordinary rule inventory is exactly the familiar intuitionistic natural-deduction one: hypothesis; $\bot$-elimination; implication introduction and elimination; conjunction introduction and the two projections; the two disjunction introductions and disjunction elimination with discharged branch assumptions; universal introduction with a fresh parameter and universal instantiation; existential introduction and elimination with a fresh witness; and equality reflexivity and substitution. There is no classical excluded-middle rule for arbitrary formulas.

Some representative derivations explain the macros. A proof of B under A yields the successor instance of $\operatorname{Sel}(y, A, B)$ under its zero instance, so derived closure introduction gives $A \to_* B$. QE at 1 gives implication elimination. For an existential, the zero gate is provable and the successor gate is the witnessed formula; DI-case and DE give the two existential rules. For conjunction, QE at 1 and 2 gives the two components because $p(1) = 1$ and $p(2) = 0$. The converse uses numerical cases and the selector endpoints.

Numerical cases for every native F are derived by DI-case at an unused witness, followed by DE and QE:

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}, \qquad z \text{ fresh.}$$

Applied to the usual disjunction of constructor cases, they give $x = 0 \vee_* \exists_* y\, (x = Sy)$. They also provide disjunction elimination for the selector macro. Universal introduction and elimination follow from the base instance and the closure.

Finally, $0 = 1$ makes every equation true: transport between the zero and successor endpoints of $C(v, s, t)$ yields $s = t$. Induction on finite formula syntax using the derived case introduction then yields every native formula. This gives falsity elimination.

6 Bi-interpretation with full first-order iPRA

6.1 The standard target theory

The target is intuitionistic first-order logic with equality in the usual primitive-recursive signature. It has 0, successor, projections, composition symbols, and a function symbol for every finite primitive recursion definition. The nonlogical axioms are their defining equations, successor separation and injectivity, and induction for every quantifier-free formula with arbitrary numerical parameters. For example, a recursive symbol F introduced from g, h satisfies

$$F(0, \vec{x}) = g(\vec{x}), \qquad F(Sn, \vec{x}) = h(n, F(n, \vec{x}), \vec{x}).$$

The induction schema is

$$(A(0,\vec{z}) \wedge \forall n(A(n,\vec{z}) \to A(Sn,\vec{z}))) \to \forall n\, A(n,\vec{z}),$$

for quantifier-free A. The repository's `PR.lean` gives this precise theory as `PRAAx`[8]. The full first-order language, not just the quantifier-free fragment, is the subject of the comparison.

6.2 The inherited scalar bridge

Let T_{eq} be ordinary intuitionistic equality logic over the iterator terms, with I_0, I_S, coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $0 = S0 \to \bot$, and induction for literal equations in arbitrary retained contexts. The pinned native development proves the following bridge[4, 5]:

1. the readings of Q, D and the native logical macros give faithful translations between the native core and T_{eq};
2. scalar terms and fixed primitive-recursive descriptions translate in both directions;
3. both term composites are provably equal to the original terms, and both formula composites are provably equivalent to the original formulas;
4. both numerical domains are whole and interpreted equality is actual equality.

This inherited theorem is the arithmetic foundation used here. The current note supplies a complete proof of the new calculus comparison; it does not claim to newly prove or simplify the entire scalar arithmetic construction. All 35 Lean source modules in the pinned ConciseIPRA tree are included as attributed reference material.

6.3 The two translations explicitly

For the forward numerical translation τ, variables, zero, and successor retain their meanings. For each fixed update term $h(u,\vec{z})$, introduce the appropriate primitive-recursive symbol

$$F_h(0,a,\vec{z}) = a, \qquad F_h(Sn,a,\vec{z}) = h^\tau(F_h(n,a,\vec{z}),\vec{z}).$$

Translate the iterator $I(n,a;u.h)$ to $F_h(n^\tau,a^\tau,\vec{z})$. Every free parameter of the update is kept fixed; nested updates are translated by recursion on their finite syntax. This is a metalevel compilation of each fixed term, not one object-level universal evaluator.

Define the forward formula reading α by

$$\begin{aligned}\alpha(s=t) &= (s^\tau = t^\tau),\\ \alpha(Q_y B) &= \alpha(B(0)) \to \forall y\, \alpha(B(y)),\\ \alpha(D_{x,y} B) &= \exists x\,(\alpha(B(x,0)) \to \forall y\, \alpha(B(x,y))).\end{aligned}$$

In the reverse numerical translation σ, projections and composition translate by term substitution. General primitive recursion is compiled into scalar iteration using a numerical pair and its projections:

$$F(n,\vec{x})^\sigma = \pi_2 I(n, \langle 0, g^\sigma(\vec{x})\rangle; u.\, \langle S\pi_1 u, h^\sigma(\pi_1 u, \pi_2 u, \vec{x})\rangle)\,. \qquad \text{(PR compilation)}$$

Pairing is an abbreviation for scalar terms supplied by the inherited bridge. For specificity, its current implementation uses

$$\langle a, b\rangle = 2^{a+b} + a, \quad \pi_1(w) = w \dot{-} 2^{\lg w}, \quad \pi_2(w) = \lg w \dot{-} \pi_1(w),$$

with addition, truncated subtraction, powers of two, and the saturating logarithm counter defined by scalar terms. The identities $\pi_1\langle a, b\rangle = a$ and $\pi_2\langle a, b\rangle = b$ are proved in `TeqPair.lean`. Only those identities are needed by (PR compilation); surjectivity onto every numerical value is not required.

Define β on ordinary iPRA formulas by applying σ to their numerical terms and replacing $\bot, \to, \wedge, \vee, \exists, \forall$ by the macros of Section 5. Equality remains numerical equality. These are precisely the roles of `NFm.toPRA` and `NFm.ofPRA` in the inherited formal development.

Theorem 6.1 (Faithfulness and both composites)**.** *For every native sequent and every iPRA sequent,*

$$\Gamma \vdash_{\mathcal{C}} A \iff \alpha(\Gamma) \vdash_{\mathrm{iPRA}} \alpha(A),$$
$$\Delta \vdash_{\mathrm{iPRA}} \varphi \iff \beta(\Delta) \vdash_{\mathcal{C}} \beta(\varphi).$$

In $\mathcal{C}$, A and $\beta\alpha(A)$ are interderivable. In iPRA, $\varphi \leftrightarrow \alpha\beta(\varphi)$ is provable. Both numerical composites have the definable identity graph $x = y$.

Proof. The pinned bridge proves every assertion with $\mathcal{N}$ in place of $\mathcal{C}$. Theorem 1.1 substitutes $\mathcal{C}$ for $\mathcal{N}$ in every sequent, including the two directions of the native round trip. No numerical domain, equality interpretation, term translation, or formula translation changes. The iPRA-side round trip is literally the inherited one. This proves all four claims simultaneously. □

Corollary 6.2 (Equiconsistency)**.**

$$\vdash_{\mathcal{C}} 0 = S0 \quad \iff \quad \vdash_{\mathrm{iPRA}} \bot.$$

Thus $\mathcal{C}$ is consistent if and only if iPRA is consistent.

The two-way faithfulness alone would not establish bi-interpretability. The composite assertions in Theorem 6.1 are indispensable. Their preservation here follows from identity of the native syntax, not from an assumption that equiconsistency implies bi-interpretability.

7 Implementation, measurements, and validation

7.1 Scope of the executable evidence

The archive includes a standalone Python implementation of the seven rules, a certificate generator and regression suite, a reproducible measurement script, frozen comparison sources, and a proposed Lean extension. The checker uses explicit term and formula constructors and checks variable scope. In particular, the passive endpoint of X is validated at the ambient scope, while its numerical template is validated with one additional placeholder. This makes the one-template interface visible in code.

The automated examples cover the computational rules, transport and its derived equality consequences, both DI branches, QE and DE, and controls for malformed syntax, invalid scope, the forbidden literal-zero source, incorrect endpoints, and invalid eigenvariable usage. The test report records 23 cases. Tests are evidence about the implementation; they are not a machine proof of soundness, completeness, or the universal bi-interpretation theorem.

7.2 Measured compact baselines

The README summarizes a slightly older frontier. The current `guarded_source` source files measure as follows under CPython 3.12.14, the version used by that project[9]:

Complete checker profile	Tokens	AST nodes	Normalized bytes	gzip-9
Token profile	257	285	568	338
AST profile	261	279	603	345
Byte profile	264	301	522	328
Matched explicit $\mathcal{M}$ control	1196	1364	3302	1005
New explicit $\mathcal{C}$ checker	1171	1339	3246	996

The coordinate minima of these three frozen comparison profiles belong to different programs. The compact baseline also assumes the canonical public-goal grammar as an input contract. A strict checker that validates this grammar has a different trusted boundary and must be compared accordingly.

The last two rows are a separate, matched experiment: both checkers have identical syntax validation, substitution, scope checks, and six other rule branches. Only the equality branch changes. The opcode is named `x` in both programs to avoid a name-length artifact. The new checker is smaller by 25 significant tokens, 25 AST nodes, and 56 normalized bytes (1.6959 percent); normalized gzip decreases by 9 bytes. The source diff and control certificates are included. In this format, the passive endpoint is read from the checked conclusion, so the equality packet needs three explicit numerical payloads instead of four.

All 25 control certificates also passed directly and in normal and optimized subprocesses; eleven certificates are byte-identical across the suites. In addition, 1,200 independent denotational checks exercised the shared substitution implementation, including insertion under binders. These checks address concrete implementation risks and are not model-theoretic tests of the universal arithmetic claims.

Measurements normalize the whole Python module using `ast.unparse(ast.parse(source))` followed by a newline. Significant tokens omit encoding, layout, comments, and end markers; AST nodes count `ast.walk`. Gzip of the normalized UTF-8 source is measured with level 9 and timestamp zero and is reported only for information. Raw and normalized figures, exact source hashes, interpreter version, and the measurement convention are recorded in the machine-readable artifacts.

The matched control establishes a modest reduction in a straightforward implementation of the current seven-rule calculus. It is not a new compact-checker record: its 3,246 bytes must not be compared as though they improved the existing 522-byte compact profile. Splitting equality away from QE can also lose sharing in that compact format. The main mathematical contribution remains the simpler equality interface and its equality-only recovery proof.

7.3 Lean status

`formalization/NativeOneSided.lean` proposes a deep embedding of the seven rules and proofs named `cprf_iff_nprf`, `cprf_iff_mprf`, `oneSided_iff_pra`, `pra_iff_oneSided`, the two round-trip theorems, and `oneSided_inconsistent_iff`. It also records the equality-only proof of unrestricted transport.

There is no Lean executable in this delivery environment. This new module has therefore *not been compiled*, and no proof-assistant verification of it is claimed. The mathematical theorem in this

note is supported by the written derivations and independent review of their rule instances. The pinned repository reports successful builds of its inherited bridge; those builds have not been rerun here. Build instructions explain how to check the extension against the pinned repository with Lean 4.32.0.

8 Alternatives screened out and further research questions

Several apparently larger reductions would require additional theorems. They are not used in the claimed result.

First, a public language with D alone and internal reusable clauses is already present in the inherited research basis. Merely removing Q from the public grammar repeats that construction. Second, forcing progress introduction at every atomic D changes the intended consequence relation; the repository already documents the obstruction $D_{x,y}(y = 1)$. Third, changing the iterator to the forward equation $I(Sn, a; h) = I(n, h(a); h)$ is not immediately reversible under fixed-parameter equation induction. The naive commutation induction would need its hypothesis at the changed seed $h(a)$. This is an unresolved proof obligation here, not an independence theorem.

The following questions are concrete next steps.

1. Can a certificate format expose the one-template transport while retaining the baseline's shared QE machinery, and beat its complete checker size under the same input contract? A positive answer needs measurements of the entire module, not just its equality branch.
2. Can the proposed Lean extension be reduced to a smaller proof interface after compiling it? The shortest mathematical comparison uses only the two-X simulation and the existing $\mathcal{M} \leftrightarrow \mathcal{N}$ theorem; the longer equality-only proof is useful but not necessary for that comparison.
3. What is the optimal certificate-size overhead of replacing two-sided transport by one-sided crossed transport? DAG sharing makes the repeated major inexpensive; a tree format needs a separate analysis.
4. Can the constant-template symmetry construction be retained with an even simpler equality rule? Any restriction to nonconstant templates must provide a replacement for this step.
5. Can the source test $s \not\equiv_{\mathrm{syn}} 0$ and witness-sensitive DI start be removed together while retaining the compact complete checker? Their current purpose is implementation dispatch, whereas the unrestricted rules are mathematically admissible.

The result is an incremental, exact simplification. It does not establish an absolutely simplest axiomatization or a significant reduction below the repository's proof-checker-size frontier. Those are stronger goals than the one-template equivalence theorem proved here.

A Reproduction and archive inventory

The archive is self-contained for Python verification and PDF rebuilding. The Lean source is an extension for the pinned upstream project. From the unpacked archive directory, run

```
python3 reproduce.py
python3 build.py
```

The first command runs the certificate tests, checks source measurements, and verifies the manifest. The second invokes pdfLaTeX for three passes, without shell escape or network access. The report source is a single standalone `report.tex` file.

Path	Purpose
`report.tex`, `report.pdf`	Mathematical statement, proofs, comparisons, and limits.
`code/`	Standalone checker, tests, and measurement utility.
`evidence/`	Measurements, test results, and source provenance.
`formalization/`	Proposed, uncompiled Lean extension and build notes.
`reference/lean/`	The 35 exact pinned native/scalar/iPRA source modules.
`reference/baselines/`	Exact comparison checker sources and their license.
`reproduce.py`, `build.py`	Offline verification and PDF rebuild entry points.
`MANIFEST.json`	SHA-256 checksums and byte counts of distributed payloads.

Rebuilding a PDF can alter its metadata and hash. Run manifest validation before rebuilding, or work on a copy. Reference-source provenance includes Git blob identifiers; the retrieved Lean bytes were checked against those identifiers. The reference code is under the repository's MIT No Attribution license, included in the archive.

References

[1] ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, pinned README. Pinned GitHub source.

[2] ProveIt contributors, `Native.lean`, native syntax, eight inference rules, and ordinary soundness. Included under `reference/lean/ConciseIPRA/` at the same pinned commit.

[3] ProveIt contributors, `NativeMinimal.lean`, the seven-rule calculus `MPrf` and `mprf_iff_nprf`. Pinned GitHub source.

[4] ProveIt contributors, `NativeBiInterpretation.lean`, the two iPRA translations, faithfulness, both round trips, and equiconsistency. Included at the same pinned commit.

[5] ProveIt contributors, `BiInterpretation.lean`, `ScalarToPRA.lean`, `PRAToScalar.lean`, `RoundTripScalar.lean`, and `RoundTripPRA.lean`. The inherited scalar/iPRA bridge, included at the same pinned commit.

[6] ProveIt contributors, `NativeReversal.lean`, and the `Research/reversal_core` package. General crossed transport and its equality-recovery argument; reviewed at the same pinned commit.

[7] ProveIt contributors, `NativeGate.lean`, `NativeLogic.lean`, and `NativeRoundTrip.lean`. Explicit native selectors, logical macros, and their round trips; included at the same pinned commit.

[8] ProveIt contributors, `PR.lean`. The precise full first-order iPRA target; included at the same pinned commit.

[9] ProveIt contributors, `Research/guarded_source/code/checker_tokens.py`, `checker_ast.py`, and `checker_bytes.py`. Exact standalone comparison programs, retrieved at the pinned commit and remeasured in this delivery; included under `reference/baselines/`.